



**FACULTY OF TECHNOLOGY AND ENGINEERING
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY
AND RESEARCH**

DEPARTMENT OF INFORMATION TECHNOLOGY

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms



PRACTICAL-2

CODE:

<https://github.com/abhishekipatadiya28-hash/DATA-STRUCTURE-ANDALGORITHM/tree/main/Practical-2>

PROBLEM-1

DIFFICULTY:

The main difficulty was understanding the concept and how to write it in code. First, I read the question properly and understood the concept. After that, I was able to write the code.

ERRORS:

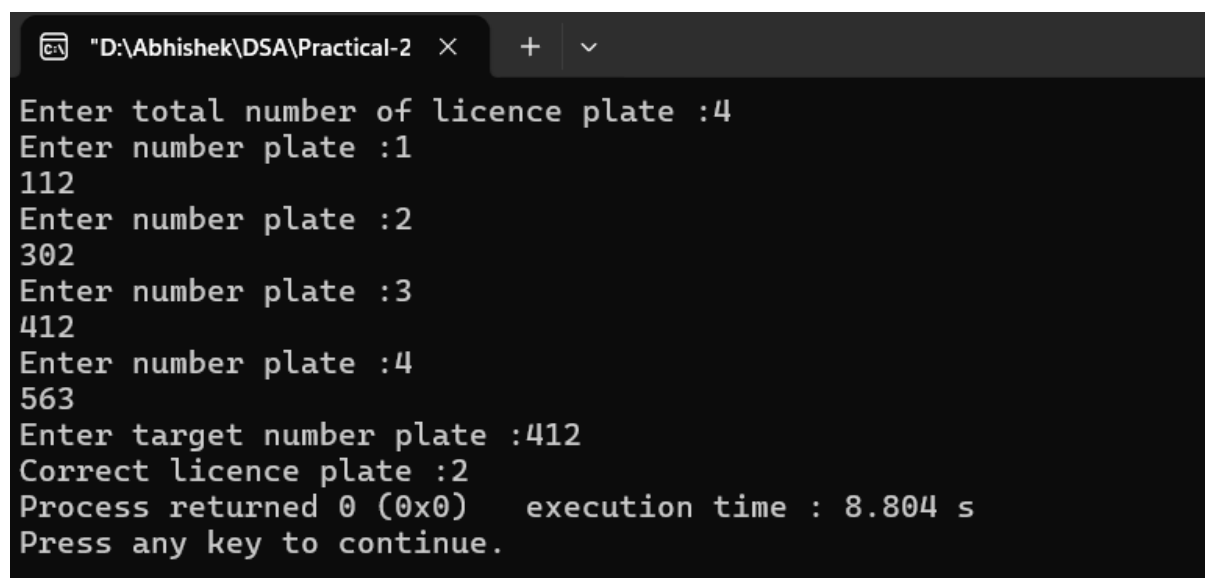
- There was no proper stopping condition.
- The function was checking only one element.
- When the target was not found, the program was not showing the correct result.

SOLUTION:

I used a simple for loop for the iterative method. For recursion, I used a stopping condition and increased the index by 1 in every recursive call.

If the target was not found, I returned -1. When the target was found, I returned it immediately so that the first occurrence was taken. I also understood the difference between index and position.

OUTPUT:



```
"D:\Abhishek\DSA\Practical-2" x + v
Enter total number of licence plate :4
Enter number plate :1
112
Enter number plate :2
302
Enter number plate :3
412
Enter number plate :4
563
Enter target number plate :412
Correct licence plate :2
Process returned 0 (0x0)    execution time : 8.804 s
Press any key to continue.
```

OUTCOMES:

- Learned the basic concept of linear search.
- Learned how to use linear search in C++.

DIFFICULTY:

The main difficulty was understanding how to use binary search with the given data. I was also confused about finding the middle element because it decides whether to search on the left or right side.

ERRORS:

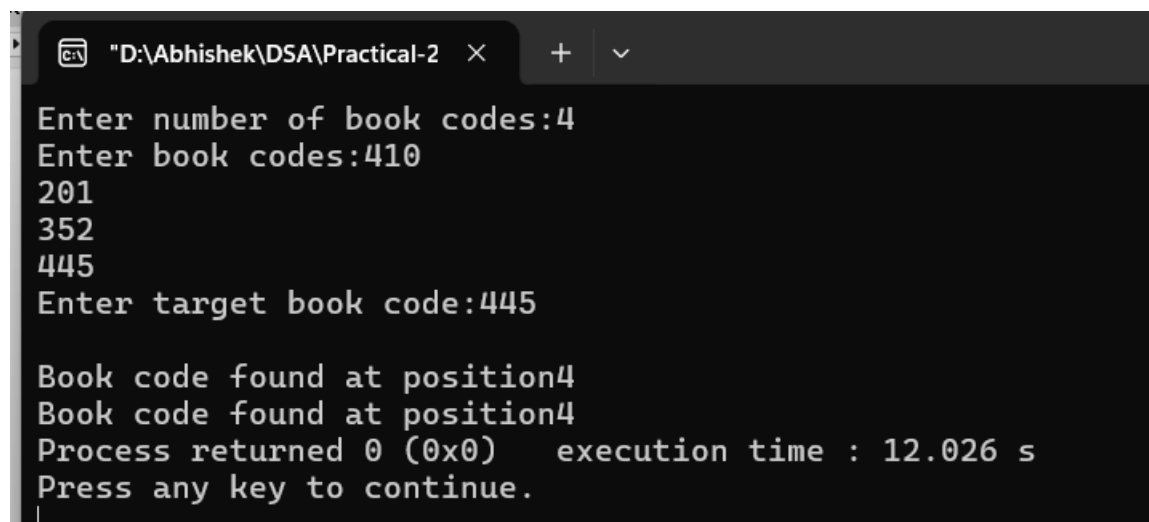
- I calculated the middle element incorrectly.
- I used the wrong loop.
- I tried to use binary search on unsorted data.

SOLUTION:

First, I set $low = 0$ and $high = n - 1$. Then I found the middle element and compared it with the target.

- If the middle value was equal to the target, the target was found.
- If the target was greater, I searched in the right half.
- If the target was smaller, I searched in the left half.

OUTPUT:



```
"D:\Abhishek\DSA\Practical-2" x + v
Enter number of book codes:4
Enter book codes:410
201
352
445
Enter target book code:445

Book code found at position4
Book code found at position4
Process returned 0 (0x0)   execution time : 12.026 s
Press any key to continue.
```

OUTCOMES:

- Learned the basic concept of binary search.
- Learned how to find the middle element.
- Understood that binary search is used only for sorted data.

KEY QUESTION:-

1. [Problem 1] If the target plate appears near the end of a very long list, your solution still checks every plate before it. Can you think of any property the input would need to have for you to skip checking some plates entirely? What does Problem 2.2 tell you about this?

ANSWER:-

If the list of plates is sorted in some known order, we may be able to skip checking many plates. For example, if the plates are sorted by their number, we can use Binary Search instead of checking every plate one by one. Binary Search checks the middle element and eliminates half of the remaining list each time. Problem 2.2 tells us about the importance of the input being sorted. If the data has a useful order, we can use that property to make the search faster. If the input list is sorted, we can skip checking some plates by using Binary Search. We can eliminate half of the remaining elements after each comparison. Problem 2.2 shows that sorted data can make searching much more efficient.

2. [Problem 2] What is the consequence of applying your solution to a list that is not sorted?

Would it always give a wrong answer, or only sometimes? Explain your reasoning without running any example.

ANSWER;-

If a searching algorithm assumes that the list is sorted, using it on an unsorted list can give a wrong answer. The list must be sorted first, or we should use a searching method that does not require sorted data, such as Linear Search.